SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	M. Mohamed Mohaiyadeen	**Reg. No.:**	192424134
Section / Batch:	2024 AI &DS	**Date:**	

Code of Conduct

I __M. Mohamed Mohaiyadeen(192424134

) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 1: Regular Expressions – Resume Information Extraction

Procedure:

1. Import the re module.
2. Store resume information in a list.
3. Extract the candidate's name using Regular Expressions.
4. Extract the email address.
5. Extract the mobile number.
6. Identify technical skills (Python, Java, SQL, Machine Learning, NLP).
7. Extract years of experience.
8. Display a structured summary of each candidate.
9. Check eligibility (minimum 2 years of experience and Python skill).
10. Display the list of eligible candidates.

Code:

```
import re

# Sample Resume Data
resumes = [
"""
Name: John Smith
Email: johnsmith@gmail.com
Phone: +91-9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
""",
```

```
"""
Name: Priya Sharma
Email: priyasharma@yahoo.com
Phone: 9123456789
Skills: Java, SQL
Experience: 1 year
""",

"""
Name: David Wilson
Email: david.wilson@outlook.com
Phone: +91 9988776655
Skills: Python, NLP, Machine Learning
Experience: 5 years
""",

"""
Name: Sneha Reddy
Email: sneha.reddy@gmail.com
Phone: 9876501234
Skills: Python, SQL
Experience: 2 years
"""
]

# Technical skills to search
technical_skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

print("========== Resume Information Extraction ==========\n")

eligible_candidates = []

for resume in resumes:

    # Extract Name
    name = re.search(r"Name:\s*(.*)", resume)
    name = name.group(1) if name else "Not Found"
```

```
    # Extract Email
    email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
    email = email.group() if email else "Not Found"

    # Extract Phone Number
    phone = re.search(r"(\+91[- ]?)?\d{10}", resume)
    phone = phone.group() if phone else "Not Found"

    # Extract Experience
    exp = re.search(r"(\d+)\s*year", resume, re.IGNORECASE)
    experience = int(exp.group(1)) if exp else 0

    # Extract Skills
    found_skills = []
    for skill in technical_skills:
        if re.search(skill, resume, re.IGNORECASE):
            found_skills.append(skill)

    # Structured Summary
    print("Candidate Summary")
    print("---------------------------")
    print("Name       :", name)
    print("Email      :", email)
    print("Phone      :", phone)
    print("Experience :", experience, "Years")
    print("Skills     :", ", ".join(found_skills))
    print()

    # Eligibility Check
    if experience >= 2 and "Python" in found_skills:
        eligible_candidates.append(name)

print("========== Eligible Candidates ==========")

if eligible_candidates:
    for candidate in eligible_candidates:
        print(candidate)
else:
    print("No eligible candidates found.")
```

Output:

Candidate Summary

Name : John Smith
Email : johnsmith@gmail.com
Phone : +91-9876543210
Experience : 3 Years
Skills : Python, Java, SQL, Machine Learning

Candidate Summary

Name: Priya Sharma
Email: priyasharma@yahoo.com
Phone: 9123456789
Experience : 1 Years
Skills: Java, SQL

Candidate Summary

Name : David Wilson
Email : david.wilson@outlook.com
Phone : +91 9988776655
Experience : 5 Years
Skills : Python, Machine Learning, NLP

Candidate Summary

Name : Sneha Reddy
Email : sneha.reddy@gmail.com
Phone : 9876501234
Experience : 2 Years
Skills : Python, SQL

========== Eligible Candidates ==========
John Smith

David Wilson
Sneha Reddy

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 2: Regular Expressions – Product Search System

Procedure:

1. Import the re module.
2. Store product names in a list.
3. Accept the search keyword from the user.
4. Perform an exact keyword search.
5. Perform a prefix search.
6. Perform a suffix search.
7. Perform a partial keyword search.
8. Perform a case-insensitive search.
9. Display all matching product names.
10. Display the total number of matching products for each search.

Code:

```
import re

# Product List
products = [
    "Python Programming Book",
    "Java Programming Book",
    "SQL Database Guide",
    "Machine Learning Basics",
    "NLP with Python",
    "Python Cookbook",
```

```
    "Wireless Mouse",
    "Bluetooth Keyboard",
    "Gaming Laptop",
    "Laptop Stand",
    "USB Keyboard",
    "Python Data Science",
    "Java Developer Guide",
    "Smart Watch",
    "Phone Charger"
]

# Function to Search Products
def search_products(pattern, description, flags=0):
    print("\n", "=" * 50)
    print(description)
    print("=" * 50)

    matches = []

    for product in products:
        if re.search(pattern, product, flags):
            matches.append(product)

    if matches:
        print("Matching Products:")
        for item in matches:
            print("-", item)
    else:
        print("No matching products found.")

    print("Total Matches:", len(matches))



# 1. Exact Keyword Search
keyword = input("Enter exact keyword: ")
search_products(r"\b" + re.escape(keyword) + r"\b",
                "Exact Keyword Search")

# 2. Prefix Search
```

```
prefix = input("\nEnter prefix: ")
search_products(r"^" + re.escape(prefix),
                "Prefix Search")

# 3. Suffix Search
suffix = input("\nEnter suffix: ")
search_products(re.escape(suffix) + r"$",
                "Suffix Search")

# 4. Partial Keyword Search
partial = input("\nEnter partial keyword: ")
search_products(re.escape(partial),
                "Partial Keyword Search")

# 5. Case-Insensitive Search
text = input("\nEnter keyword for case-insensitive search: ")
search_products(re.escape(text),
                "Case-Insensitive Search",
                re.IGNORECASE)
```

Sample Input:

Enter exact keyword: Python

Enter prefix: Python

Enter suffix: Book

Enter partial keyword: Laptop

Enter keyword for case-insensitive search: python

Sample Output

```
==================================================
Exact Keyword Search
==================================================
Matching Products:
- Python Programming Book
```

- NLP with Python
- Python Cookbook
- Python Data Science
Total Matches: 4

==========
Prefix Search
==========
Matching Products:
- Python Programming Book
- Python Cookbook
- Python Data Science
Total Matches: 3

==========
Suffix Search
==========
Matching Products:
- Python Programming Book
- Java Programming Book
Total Matches: 2

==========
Partial Keyword Search
==========
Matching Products:
- Gaming Laptop
- Laptop Stand
Total Matches: 2

==========
Case-Insensitive Search
==========
Matching Products:
- Python Programming Book
- NLP with Python
- Python Cookbook
- Python Data Science
Total Matches: 4

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Section 3: Regular Expressions – University Registration System

Procedure:

1. Import the re module.
2. Accept the student's register number.
3. Accept the institutional email address.
4. Accept the course code.
5. Accept the semester.
6. Accept the mobile number.
7. Validate each field using Regular Expressions.
8. Display validation messages.
9. Generate the final registration status.
10. Display whether the registration is successful or failed.

Code:

```
import re

print("========== University Registration System ==========")

# User Input
reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester (1-8): ")
mobile = input("Enter Mobile Number: ")

status = True
```

```
# 1. Register Number Validation
# Format: 23CS1010 (2 digits + 2 letters + 4 digits)
if re.fullmatch(r"\d{2}[A-Z]{2}\d{4}", reg_no):
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")
    status = False

# 2. Institutional Email Validation
# Example: student@university.edu
if re.fullmatch(r"[a-zA-Z0-9._%+-]+@university\.edu", email):
    print("Institutional Email: Valid")
else:
    print("Institutional Email: Invalid")
    status = False

# 3. Course Code Validation
# Format: CS101, MA202, AI305
if re.fullmatch(r"[A-Z]{2,3}\d{3}", course_code):
    print("Course Code: Valid")
else:
    print("Course Code: Invalid")
    status = False

# 4. Semester Validation (1 to 8)
if re.fullmatch(r"[1-8]", semester):
    print("Semester: Valid")
else:
    print("Semester: Invalid")
    status = False

# 5. Mobile Number Validation
# 10-digit Indian mobile number starting with 6-9
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")
    status = False
```

```
# Final Registration Report
print("\n========== Registration Status ==========")

if status:
    print("Registration Successful")
else:
    print("Registration Failed")
    print("Please correct the invalid fields and try again.")
```

Sample Input:

Enter Register Number: 23CS1010
Enter Institutional Email: student@university.edu
Enter Course Code: CS301
Enter Semester (1-8): 5
Enter Mobile Number: 9876543210

Sample Output:

========== University Registration System ==========

Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

========== Registration Status ==========
Registration Successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____